

WhatsApp Business API — Embedded Signup

Day 1 Learning Summary & Implementation Reference

Author: Rajkumar **Date:** June 9, 2026 **Stack:** React (frontend) + Node.js (backend) **Scope today:** Frontend + initial backend

Status: In progress (Day 1)

What I covered today: the full **frontend** Embedded Signup flow (Facebook SDK, session listener, launch & response callbacks) and the **start of the backend** (token-exchange endpoint, webhook receiver). Persistence/database is **not built yet** — token storage and data sync are noted below as next steps.

1. What I Learned — The Core Idea

Embedded Signup is a secure OAuth flow that lets a customer connect their WhatsApp Business account to the Cloud API *without ever leaving your application*. Rather than redirecting to Facebook, the user authenticates inside an SDK-launched dialog and your app receives a short-lived **authorization code**, which the backend exchanges for a long-lived **access token**.

Why it matters

- Seamless UX — no external redirects
- Secure server-side token handling
- Supports multiple WhatsApp accounts (multi-WABA)
- Optional **Coexistence** with the WhatsApp Business App

Coexistence Mode

Lets a business run the **WhatsApp Business App** (manual 1:1 chats) and the **Cloud API** (automation) *at the same time*, with message & history synchronization between them. It requires a special signup configuration (see §4).

2. How the Flow Works (Architecture)

Browser User clicks “Sign up with WhatsApp” → FB SDK opens login dialog → user authorizes → receives **code** (30s TTL) → sent to backend

Backend Receives code → exchanges for **Access Token + WABA ID** → stores encrypted → starts sync (if coexistence)

Webhooks Meta pushes **messages** , **account_update** , **history** , etc. → process async → respond to customers

The whole exchange hinges on speed: the authorization code is valid for only **30 seconds**, so the frontend must forward it and the backend must redeem it with *zero* delay.

3. Key Configuration Values

Parameter	Example	Where to Find It
App ID	2140782060178223	App Dashboard
App Secret	abc123...def456	App Settings > Basic
Config ID	1731129688347511	Facebook Login > Configurations
WABA ID	524126980791429	Session event listener (post-signup)
Phone Number ID	106540352242922	Session event listener (post-signup)
Business ID	2729063490586005	Session event listener (post-signup)

4. Standard vs. Coexistence Configuration

Standard Mode

```
const config = {
  config_id: "YOUR_CONFIG_ID",
  response_type: "code",
  override_default_response_type: true,
  extras: { setup: {} }
};
```

Coexistence Mode

```
const config = {
  config_id: "YOUR_CONFIG_ID",
  response_type: "code",
  override_default_response_type: true,
  extras: {
    setup: {},
    featureType:
      "whatsapp_business_app_onboarding",
    sessionInfoVersion: "3"
  }
};
```

Key difference: Coexistence adds `featureType` + `sessionInfoVersion: "3"`, unlocking simultaneous use of both apps with message syncing. It also caps sending at **20 messages/second**.

5. The Critical Code — Immediate Token Exchange

⚠ 30-Second TTL. The single most important lesson: the authorization code expires in 30 seconds. No `await` on slow work, no user prompts, no batching — forward and exchange it *immediately*, or you get **Error 190 “Code expired.”**

Frontend — forward instantly

```
const fbLoginCallback = (response) => {
  if (response.authResponse) {
    const code = response.authResponse.code;
    fetch('/api/auth/exchange-code', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ code })
    }).catch(e =>
      console.error('Exchange failed:', e));
  }
};
```

Backend — redeem instantly

```
app.post('/api/auth/exchange-code',
  async (req, res) => {
    const { code } = req.body;
    const r = await axios.post(
      'https://graph.facebook.com/v25.0/oauth/access_token',
      { client_id: process.env.APP_ID,
        client_secret: process.env.APP_SECRET,
        redirect_uri: process.env.REDIRECT_URI,
        code });
    const { access_token, user_id } = r.data;
    // TODO: persist token securely
    // (DB not built yet — Day 1)
    res.json({ success: true });
  });
```

6. Webhook Integration

Webhooks must use HTTPS, verify the `X-Hub-Signature-256` signature, respond **200 immediately**, then process asynchronously. The six event types I studied (full handlers are a next step):

Event	Purpose	Handler Action
<code>account_update</code>	Account connected / disconnected	Update business status
<code>messages</code>	Customer sends a message	Save & route message

message_status	Delivery / read updates	Update message status
history	Chat history sync (coexistence)	Store message history
smb_app_state_sync	Contact sync (coexistence)	Sync contacts
smb_message_echoes	Messages sent from the Business App	Mirror to Cloud API

```
// Verify (GET) then handle (POST) – respond fast, process later
app.get('/webhook', (req, res) => {
  req.query['hub.verify_token'] === process.env.WEBHOOK_VERIFY_TOKEN
    ? res.send(req.query['hub.challenge']) : res.sendStatus(403);

  app.post('/webhook', express.json(), (req, res) => {
    res.status(200).send('ok'); // ACK immediately
    setImmediate(() => req.body.entry?.forEach(e => {
      e.changes?.forEach(({ field, value }) => route(field, value))); // process async
    }));
  });
});
```

7. The 5 Problems I Hit (and How I Solved Them)

Issue	Cause	Solution
“Code expired” (Error 190)	Code not exchanged within 30s	Exchange immediately, no delays
“Domain not allowed”	Domain missing from dashboard	Add to <i>Allowed Domains</i> + <i>Valid OAuth Redirect URIs</i>
SDK not loading	Network / domain / HTTP	Serve over HTTPS, verify domain config in app
Coexistence not appearing	Wrong config parameters	Set <code>featureType</code> + <code>sessionInfoVersion: "3"</code>
Webhooks silent	Fields not subscribed	WhatsApp > Configuration > Webhooks → subscribe all fields

8. Environment & Security

Environment Variables (used so far)

```
FACEBOOK_APP_ID=your_app_id
FACEBOOK_APP_SECRET=your_app_secret
FACEBOOK_CONFIG_ID=your_config_id
REDIRECT_URI=https://you.com/callback
WEBHOOK_VERIFY_TOKEN=your_token
```

Security Notes

- HTTPS everywhere (front, back, webhooks)
- Verify `X-Hub-Signature-256` on webhooks
- Secrets only in env vars, never in code
- Rate limit: 20 msg/s (coexistence)
- Validate all input; use CSRF protection
- *Next step*: encrypt & persist access tokens once a DB is added

9. What I'll Do Next

- Add a database to **persist access tokens** + WABA / phone-number IDs securely.
- Implement the **webhook event handlers** (messages, account_update, status).
- Build **coexistence sync** (contacts, history, message echoes) once persistence exists.
- Add token **refresh** before expiry and basic **monitoring**.

10. Key Takeaways

- **30-second TTL is everything** — exchange the authorization code immediately.
- **Coexistence needs special config** — distinct `featureType` & `sessionInfoVersion` .
- **Webhooks must be fast** — ACK 200 first, process asynchronously.
- **Security first** — HTTPS everywhere and verified webhook signatures.
- **Today's milestone** — working frontend signup flow + initial token-exchange backend.

Reference docs: Implementation & onboarding guides at developers.facebook.com/documentation/business-messaging/whatsapp/embedded-signup ; Webhooks & Cloud API reference under the same path.